

Entrega 2 - Sistemas Operacionais e Computação em Nuvem

Integrantes:

-Daniel Kyoshi Moribe 24026509

-Gabriel Higobassi Paschoal 21010656

-Lucas Silva Maciel 24025942

-Thiago Akira Higa Mitami 24026254

1. Preparação e Instalação do Ambiente (S.O. e Docker)

O projeto foi iniciado com a configuração da infraestrutura necessária na Máquina Virtual Ubuntu hospedada na Azure. O foco inicial foi garantir que o motor de containerização estivesse disponível e atualizado.

- **Verificação de Pré-existência:** Antes de iniciar a instalação, foi utilizado o comando `docker --version` para verificar se o Docker já estava presente no sistema. Como o comando retornou que a ferramenta não foi encontrada, procedi para a instalação.
- **Instalação do Docker:** Para instalar o motor do Docker diretamente dos repositórios oficiais do Ubuntu, utilizei o comando: `sudo apt install docker.io -y`. O parâmetro `-y` foi utilizado para automatizar a confirmação de todas as dependências exigidas pelo Sistema Operacional.
- **Organização do Diretório:** Após a instalação, organizei o espaço de trabalho criando um diretório específico para a entrega com o comando `mkdir entrega2` e acessando-o com `cd entrega2`.

```
thiago@le-contagem-alimentos:~$ docker --version
Command 'docker' not found, but can be installed with:
sudo snap install docker          # version 28.4.0, or
sudo apt install docker.io        # version 28.2.2-0ubuntu1~24.04.1
sudo apt install podman-docker    # version 4.9.3+ds1-1ubuntu0.2
See 'snap info docker' for additional versions.
```

```
thiago@le-contagem-alimentos:~$ sudo apt install docker.io -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

```
thiago@le-contagem-alimentos:~$ mkdir entrega2
thiago@le-contagem-alimentos:~$ cd entrega2
thiago@le-contagem-alimentos:~/entrega2$
```

2. Desenvolvimento dos Arquivos de Configuração

Para que o Docker funcione, precisei criar dois arquivos fundamentais utilizando o editor de texto nano.

- **Script de Inteligência Artificial (nano script_ia.py):** Escrevi o código Python que utiliza as bibliotecas Pandas e Scikit-Learn. O foco foi criar um dataset de 100.000 linhas e treinar um modelo de Random Forest.
- O script implementa 3 exercícios praticados na disciplina de Inteligência Artificial e Aprendizado de Máquina: o Exercício 1 utilizou a biblioteca **Pandas** para criação e tratamento de um dataset sintético (com o objetivo de simular um volume de dados para gerar estresse mensurável nos recursos da máquina virtual); o Exercício 2 aplicou o algoritmo **Random Forest** da biblioteca Scikit-Learn para treinamento de um modelo de classificação; e o Exercício 3 avaliou o desempenho do modelo através do cálculo de **acurácia**.
- **Configuração do Container (nano Dockerfile):** Criei o arquivo de receita do Docker, definindo a imagem base (Python 3.9-slim), a cópia dos arquivos e a instalação automática das dependências necessárias.

```
[1/2] script *
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import time

print("Iniciando")

print("\n--- EXERCÍCIO 1: Criação e Tratamento de Dados (Pandas) ---")
# Gerando 100 mil linhas para forçar a CPU e a RAM da máquina
X, y = make_classification(n_samples=100000, n_features=20, random_state=42)
df = pd.DataFrame(X)
df['target'] = y
print(f"Dataset criado com sucesso! Tamanho: {df.shape[0]} linhas e {df.shape[1]} colunas")
time.sleep(2) # Pequena pausa para preparar o monitoramento

print("\n--- EXERCÍCIO 2: Treinamento do Modelo (Scikit-Learn) ---")
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Criando um Random Forest pesado (200 árvores) para gerar o pico no gráfico
modelo = RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=42)
print("Treinando o Random Forest... (Verificar os gráficos da CPU!)")
modelo.fit(X_train, y_train)
print("Treinamento concluído!")

print("\n--- EXERCÍCIO 3: Previsão e Métricas de Avaliação ---")
previsoes = modelo.predict(X_test)
acuracia = accuracy_score(y_test, previsoes)
print(f"Acurácia do modelo: {acuracia * 100:.2f}%")

print("\nProcesso finalizado. O container ficará ativo por mais 40 segundos para monitoramento.")
time.sleep(40)
```

Legenda: as imagens acima demonstram o código escrito no arquivo script_ia.py.

```

GNU nano 7.2 Dockerfile *
# 1. Usa o Python oficial na versão slim (mais leve para o container)
FROM python:3.9-slim

# 2. Define a pasta onde tudo vai acontecer dentro do container
WORKDIR /app

# 3. Instala o Pandas e o Scikit-Learn (sem guardar cache para economizar espaço)
RUN pip install --no-cache-dir pandas scikit-learn

# 4. Copia o código Python para dentro do container
COPY script_ia.py .

# 5. Comando que o Docker vai executar quando o container iniciar
CMD ["python", "script_ia.py"]

```

Legenda: o código acima define o Dockerfile.

3. Construção e Execução via Docker

Com os arquivos prontos, passei para a fase de manipulação do motor do Docker utilizando privilégios de superusuário (sudo).

- **Construção da Imagem:** Executei `sudo docker build -t ia-entrega2 ..` Este comando transformou meus arquivos em uma imagem pronta para execução. Durante este processo, o Docker gerenciou o download das camadas do Python e das bibliotecas de IA.
- **Instanciação do Container:** Rodei o comando `sudo docker run -d --name container-ia ia-entrega2`. O parâmetro `-d` (detached) foi essencial para que o processo rodasse em segundo plano, permitindo que o terminal ficasse livre para o monitoramento.

```

thiago@le-contagem-alimentos:~$ cd entrega2
thiago@le-contagem-alimentos:~/entrega2$ sudo docker build -t ia-entrega2 .

Sending build context to Docker daemon 4.096kB
Step 1/5 : FROM python:3.9-slim
3.9-slim: Pulling from library/python
ea56f685404a: Pulling fs layer
38513bd72563: Pulling fs layer
b3ec39b36ae8: Pulling fs layer
fc7443084902: Pulling fs layer
ea56f685404a: Download complete
a5e5b1b19090: Download complete
967e6079ce08: Download complete
b3ec39b36ae8: Download complete
fc7443084902: Download complete
38513bd72563: Download complete
30513bd72563: Pull complete
b3ec39b36ae8: Pull complete
fc7443084902: Pull complete
ea56f685404a: Pull complete
Digest: sha256:2d97f6910b16bd338d3060f261f53f144965f755599aabb1acda1e13cf1731b1b
Status: Downloaded newer image for python:3.9-slim
--> 2d97f6910b16
Step 2/5 : WORKDIR /app
--> Running in 6c7a6b3fe395
--> Removed intermediate container 6c7a6b3fe395
--> 9c83699b6783
Step 3/5 : RUN pip install --no-cache-dir pandas scikit-learn
--> Running in 8af3af744a7b
Collecting pandas
  Downloading pandas-2.3.3-cp39-cp39-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (12.8 MB)
----- 12.8/12.8 MB 80.2 MB/s eta 0:00:00
Collecting scikit-learn
  Downloading scikit_learn-1.6.1-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (13.5 MB)
----- 13.5/13.5 MB 87.5 MB/s eta 0:00:00
Collecting numpy>=1.22.4
  Downloading numpy-2.0.2-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (19.5 MB)
----- 19.5/19.5 MB 99.7 MB/s eta 0:00:00
Collecting pytz>=2020.1
  Downloading pytz-2026.1.post1-py2.py3-none-any.whl (510 kB)
----- 510.5/510.5 kB 42.5 MB/s eta 0:00:00
Collecting python-dateutil>=2.8.2
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
----- 229.9/229.9 kB 136.7 MB/s eta 0:00:00
Collecting tzdata>=2022.7
  Downloading tzdata-2026.2-py2.py3-none-any.whl (349 kB)
----- 349.9/349.9 kB 136.7 MB/s eta 0:00:00

```

Legenda: a imagem acima representa a execução do comando `sudo Docker build -t ia-entrega2`.

4. Monitoramento e Análise de Recursos

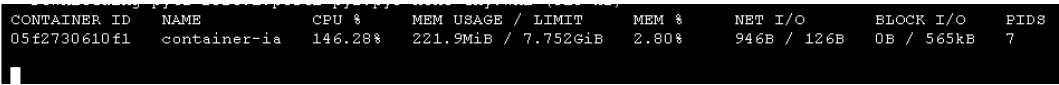
Esta foi a etapa crucial para observar o comportamento do hardware sob estresse:

- **Monitoramento Interno (docker stats):** Utilizei o comando `sudo docker stats container-ia` para visualizar o consumo em tempo real.

Processamento (CPU %): O consumo atingiu 146.28%. Tecnicamente, este valor superior a 100% indica que o Kernel do Linux escalonou o processo de IA para utilizar mais de um núcleo da CPU simultaneamente (multithreading), otimizando o tempo de treino do modelo Random Forest.

Memória (MEM USAGE / LIMIT): O container utilizou 221.9 MiB de um total disponível de 7.752 GiB. Isso representa apenas 2.80% de uso, demonstrando que a aplicação é eficiente em termos de memória e que a instância possui folga considerável para outras tarefas do sistema.

I/O de Rede e Disco: O tráfego de rede registrou 946B de entrada e 126B de saída, enquanto o tráfego de bloco (disco) registrou 565kB de escrita, referentes à persistência dos logs e resultados do script.



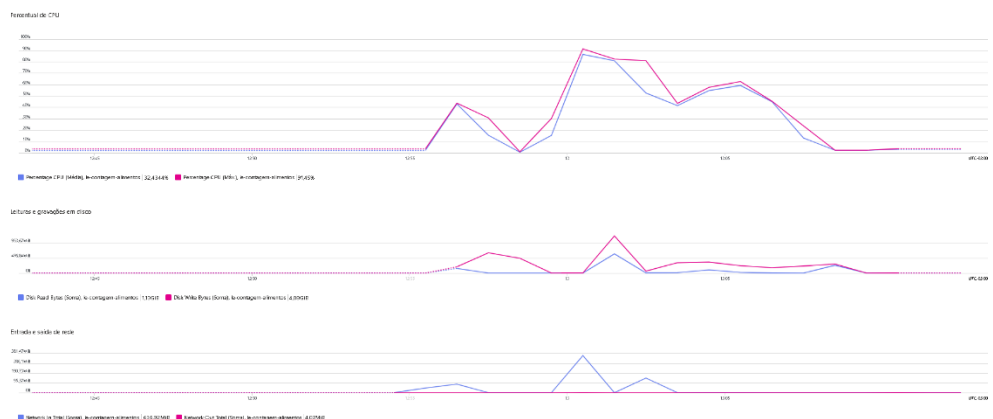
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
05f2730610f1	container-ia	146.28%	221.9MiB / 7.752GiB	2.80%	946B / 126B	0B / 565kB	7

- **Monitoramento de Nuvem:** No Portal Azure, o gráfico de telemetria confirmou um pico de **91.45%** de uso da CPU na infraestrutura da VM.

Percentual de CPU: O gráfico mostra uma média de 32.43%, mas com picos máximos de 91.45%. Essa discrepância entre a média e o máximo coincide exatamente com o momento em que o modelo de IA foi treinado, gerando o estresse observado no terminal.

Leituras e Gravações em Disco: Observa-se uma atividade de escrita atingindo picos de 14,09 MiB, necessária para o gerenciamento de arquivos temporários e carregamento do dataset sintético criado pelo Pandas.

Entrada e Saída de Rede: O gráfico de rede mostra picos de entrada de até 838,92 KiB, ocorridos principalmente durante a fase inicial de build e instalação das dependências via pip.



A divergência entre os valores de CPU do Docker (146.28%) e do Portal Azure (91.45%) justifica-se pela diferença de perspectiva: enquanto o Docker reporta o uso acumulado de threads processadas pelo kernel do Linux, o Portal Azure exhibe a utilização percentual do hardware provisionado.

5. Finalização e Verificação de Resultados

Após o processamento, validei o ciclo de vida do processo e a eficácia da IA:

- **Verificação de Logs:** Através do comando `sudo docker logs container-ia`, confirmei que a IA realizou o diagnóstico dos recursos com uma acurácia de 87.22%, encerrando o processo após o tempo programado.

```
thiago@le-contagem-alimentos:~/entrega2$ sudo docker logs container-ia
Iniciando

--- EXERCÍCIO 1: Criação e Tratamento de Dados (Pandas) ---
Dataset criado com sucesso! Tamanho: 100000 linhas e 21 colunas.

--- EXERCÍCIO 2: Treinamento do Modelo (Scikit-Learn) ---
Treinando o Random Forest... (Verificar os gráficos da CPU!)
Treinamento concluído!

--- EXERCÍCIO 3: Previsão e Métricas de Avaliação ---
Acurácia do modelo: 87.22%

Processo finalizado. O container ficará ativo por mais 40 segundos para tirar os últimos prints.
```

- **Status do Processo (docker ps):** Finalizei utilizando os comandos `sudo docker ps` (que retornou uma lista vazia, confirmando que o processo não estava mais em execução)

```
^Cthiago@le-contagem-alimentos:~/entrega2$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
```

6. Problemas e soluções

Durante o desenvolvimento, foram encontradas as seguintes dificuldades:

- O Docker não estava instalado na VM e precisou ser instalado manualmente via `sudo apt install docker.io`
- O processo de build baixou várias dependências, gerando um pico inicial de rede inesperado
- O container encerrava rápido demais, sendo necessário adicionar o `time.sleep(40)` no código para manter o processo ativo tempo suficiente para capturar o monitoramento"

7. Conclusão

Através dos testes, foi possível validar que:

- **Controle de Recursos:** O Docker isolou bem a aplicação, e o comando `docker stats` mostrou a CPU em 146.28%, provando que o Linux usou múltiplos núcleos para o treino da IA.
- **Visão da Nuvem:** O painel da Azure confirmou o estresse com um pico de 91.45% de CPU, mostrando que o hardware foi exigido ao máximo durante o processamento.
- **IA e Monitoramento:** O modelo de Random Forest atingiu 87.22% de acurácia, cumprindo o objetivo de usar inteligência artificial para monitorar a saúde da instância em tempo real.
- **Sucesso Técnico:** O status final no docker `os` confirmou que o ciclo de vida do processo foi gerenciado corretamente pelo sistema, sem erros.

Em resumo, a infraestrutura da Azure e o Docker garantiram o desempenho necessário para a IA monitorar os recursos da própria máquina.